

The V3 example game, annotated

Complete source, with explanations beside the code in execution order.

What you will read. The Crate Collector in `examples/game/game.cpp`: a camera moves across one grass plane, crates fall and can be collected, a score overlay appears, and music plays. All 324 source lines are printed, followed by the complete game CMake file.

Read	Follow
Pages 3-10: helpers	Imports, constants, Crate state, sphere geometry and scene construction.
Pages 11-15: startup	Arguments, engine/window construction, services, audio, camera and retained state.
Page 16: GUI	Registration and execution of the score/music callback.
Pages 17-21: frames	Timing, steering, spawning, gravity, engine stepping and collection.
Pages 22-23: build	The complete CMake target, music preparation and startup tests.

How to use the annotations. Left: the original numbered code. Right: line ranges, objects and calls explained in plain language. Declarations come first in the file, but execution begins in `main`; its call to `loadScene` enters the earlier helper. Repeated vector/expected/handle patterns are explained once and then connected to their particular use.

Printing conventions. Source line numbers are preserved. Tabs are expanded; unusually long alignment indentation is shortened for print. Long lines wrap with no repeated number. Statements and comments are retained; no ellipses replace source. Exact original `game.cpp` and `CMakeLists.txt` are also embedded as PDF attachments.

Source edition: 6 October 2026, branch V3, commit 377b28c1943c6ea69e884d5909986b3a43149326.
Repository: `C:/data/GitHub/ViennaVulkanEngine`. This is a source walkthrough; creating the PDF does not modify the game or rebuild it.

Who owns what?

Recognize owners, borrowed services, small values and callbacks before stepping.

Object or type	What exists in this game
EngineBuilder<> / WindowSetup	Temporary setup helpers. build() creates the persistent Engine; init() opens its windows.
vve::Engine<>	The owner of the selected implementation, subsystem wrappers and lifetime. No user systems are registered.
World / RenderSystem / AudioSystem / GuiSystem / WindowSystem / InputState	Reference views into that Engine. Copying wrappers shares its state; it does not duplicate GPU or audio resources.
Crate / vector<Crate>	One game-state record per crate / the owner of the active records, including position, velocity and landed flag.
DefaultCameraController / Camera	Persistent movement/angle state / a view description published to RenderSystem each frame.
Vec2, Vec3, Position, Direction, Transform, LinearColor, PixelExtent, DeltaTime	Values describing dimensions, geometry, location, light, placement, color, window size and time.
RenderObjectHandle / SoundHandle / AudioPlaybackHandle	Typed IDs for a visible placement / reusable sound data / one playback voice. IDs do not own resources.
expected<T,Error> / unexpected	A successful value or an error. Check before *result; unexpected propagates failure.
array / Vector / filesystem::path / string_view	Fixed lists / std::vector-backed dynamic arrays / file locations / borrowed command-line text.
mt19937 / random_device / uniform_real_distribution / steady_clock	Random engine / seed source / range mapping / elapsed-time source used by main.
startMusic / GUI lambda / erase_if predicate	Stored local function / registered per-frame callback / immediate collection callback. Each captures existing game state.

One iteration: read last-pollled input -> update camera and crates -> engine.step polls fresh input and draws (including GUI) -> check stop/Escape -> collect landed crates. Removal and the new score normally appear in the next picture.

Imports and asset names

game.cpp | source lines 1-19 | explanations refer to these numbers

```
1 #include <imgui.h>
2
3 import std;
4 import VVEngine;
5 import VVE.ExampleSupport;
6
7 /**
8  * @file
9  * @brief Crate-collector mini game built entirely on the public VVE facade.
10  *
11  * The player steers the standard camera controller across a grassy plane while
12  * crates rain from the sky. Driving the camera onto a crate collects it for a
13  * point. The score is shown in an ImGui overlay in the top-left corner.
14  */
15 namespace {
16
17 constexpr auto grassTextureRelativePath = "assets/game/plane/grass.jpg"; ///< Tiled ground texture.
18 constexpr std::array musicFiles{"dance.mp3", "ophelia.mp3", "getout.ogg"}; ///< Original V2 soundtrack.
19 constexpr std::array crateTextureRelativePaths{"assets/game/crate0/diffuse.png", "assets/game/crate1/diffuse.png"}; ///< Wood crate textures, alternated per spawn.
```

1-5 | Headers and modules

imgui.h declares the ordinary Dear ImGui functions. import std supplies standard C++ facilities; import VVEngine supplies the public vve API. VVE.ExampleSupport supplies the example-only assetRoot and frameLimit helpers.

7-15 | The game and its private helpers

The opening comment describes the game. The anonymous namespace keeps helper names local to this translation unit. main later remains the application entry point.

17-19 | Fixed asset lists

grassTextureRelativePath is the ground image. Two std::array objects keep three music filenames and two crate textures. CTAD infers their element types and sizes. These names refer to files, not loaded textures or sounds.

What gets instantiated here?

Only constant path strings and arrays exist at this stage. No window, renderer, model or audio device has been opened. Constants are used by the later scene and startup calls.

Sizes, timing and lighting constants

game.cpp | source lines 20-36 | explanations refer to these numbers

```

20
21 constexpr float groundHalfExtent = 20.0F;    ///< Half side length of the square play
    field in metres.
22 constexpr float groundTileSize = 4.0F;       ///< Metres covered by one repeat of the
    grass texture.
23 constexpr float eyeHeight = 1.0F;            ///< Fixed camera height above the plane
    in metres.
24 constexpr float collectRadius = 1.2F;        ///< Horizontal distance at which the cam
    era collects a crate.
25 constexpr float crateHalfSize = 0.5F;        ///< Half side length of a falling crate
    cube.
26 constexpr float crateRestY = 0.5F;           ///< Crate centre height once landed; the
    bottom rests on y=0.
27 constexpr float crateSpawnY = 24.0F;         ///< Height crates spawn at before fallin
    g.
28 constexpr float gravity = 12.0F;              ///< Downward acceleration applied to fal
    ling crates (m/s^2).
29 constexpr float spawnInterval = 1.3F;        ///< Seconds between crate spawns while b
    elow the cap.
30 constexpr std::size_t maxActiveCrates = 12U;  ///< Maximum simultaneously present crate
    s.
31 constexpr std::uint32_t windowHeight = 960U;  ///< Window and camera target width in pi
    xels.
32 constexpr std::uint32_t windowHeight = 540U;  ///< Window and camera target height in p
    ixels.
33 constexpr vve::Vec3 sunLightDirection{0.35F, -1.0F, 0.26F}; ///< Light-to-scene directio
    n of the sun's light (normalised at use).
34 constexpr float sunDistance = 80.0F;          ///< Camera-relative distance to the sun;
    must stay inside the renderer's 100 m far plane.
35 constexpr float sunRadius = 1.6F;             ///< Sun sphere radius; with sunDistance
    this spans roughly 2.3 degrees.
36

```

21-30 | Rules for the world

Half extent 20 means a 40 by 40 ground plane. A texture repeat covers 4 units. Crates have half size 0.5, so a center at y=0.5 puts their bottom at y=0. They spawn at y=24, fall with acceleration 12 and are collected within horizontal radius 1.2.

29-32 | Limits and display

A crate spawns every 1.3 seconds while fewer than 12 are active. The initial window is 960 by 540. The game uses its own timer and manual gravity; these constants do not install a physics system.

33 | vve::Vec3

This vector describes the direction light travels toward the scene. Its negative Y component points downward; X and Z tilt it. Later normalize makes it a unit direction.

34-35 | The visible sun

The sphere has radius 1.6 and stays 80 units from the camera. Camera defaults have far distance 100, so the sphere fits that range. These are game choices, not universal renderer limits.

Crate: gameplay state and render identity

game.cpp | source lines 37-44 | explanations refer to these numbers

```
37  /// @brief One falling or landed crate tracked by the game loop.
38  struct Crate {
39      vve::RenderObjectHandle handle{}; ///< Live render-object handle used to move or remove the crate.
40      vve::Vec3 position{};             ///< Current world position of the crate centre.
41      float velocityY{};                ///< Vertical velocity while falling.
42      bool landed{};                   ///< True once the crate has reached the ground.
43  };
44
```

38 | The Crate struct

Each Crate is a small game-owned record. main creates instances when spawning and stores them in `std::vector<Crate>`. A Crate does not own a renderer or GPU buffer.

39 | RenderObjectHandle

This typed ID names one visible object owned by the engine. The game sends it to `setObjectTransform` or `removeObject`. Retaining the handle alone does not keep a removed object alive.

40-42 | Position, speed and landed state

`Vec3` holds the current center. `velocityY` is the vertical speed, initially zero. `landed` becomes true when the bottom reaches the plane. These values make the rules independent of the rendering resource.

The connection to the picture

The loop advances this record, then publishes its position through a Transform. Collection must remove the render object and erase the Crate record. Neither side automatically removes the other.

makeSphere: generate vertex positions

game.cpp | source lines 45-57 | explanations refer to these numbers

```
45  /// @brief Builds a UV-sphere triangle mesh in object space for the sun.
46  void makeSphere(float radius, std::uint32_t stacks, std::uint32_t slices,
47                  vve::Vector<vve::Vec3> &positions, vve::Vector<std::uint32_t> &indices)
48  {
49      constexpr float pi = std::numbers::pi_v<float>;
50      // Generate one ring of vertices per stack from the north to the south pole.
51      for (std::uint32_t stack = 0; stack <= stacks; ++stack) {
52          const float phi = pi * static_cast<float>(stack) / static_cast<float>(stacks);
53          for (std::uint32_t slice = 0; slice <= slices; ++slice) {
54              const float theta = 2.0F * pi * static_cast<float>(slice) / static_cast<float>(slices);
55              positions.push_back(vve::Vec3{radius * std::sin(phi) * std::cos(theta), radius * std::cos(phi),
56              radius * std::sin(phi) * std::sin(theta)});
57          }
```

46-47 | The function and output vectors

makeSphere fills two caller-owned Vector objects by reference. vve::Vector<T> is std::vector<T>. Vec3 is a point in the sun mesh; the integer vector will hold triangle indices. radius, stacks and slices control size and tessellation.

48-53 | Angles and rings

std::numbers::pi_v<float> supplies pi. static_cast<float> makes the divisions floating point. phi goes from north pole to south pole; theta makes a full circle for each ring.

54-55 | push_back and trigonometry

sin and cos convert the angles into Cartesian XYZ coordinates. Constructing Vec3 stores those coordinates; push_back appends the point to the CPU vector. This calculation does not contact the GPU.

Inclusive loop bounds

Both loops include their final index, duplicating the seam and pole positions. With 24 stacks and 32 slices, the later call generates $25 \times 33 = 825$ positions.

makeSphere: connect the triangle indices

game.cpp | source lines 58-70 | explanations refer to these numbers

```
58     // Connect adjacent rings into two triangles per quad cell.
59     const std::uint32_t ringStride = slices + 1U;
60     for (std::uint32_t stack = 0; stack < stacks; ++stack) {
61         for (std::uint32_t slice = 0; slice < slices; ++slice) {
62             const std::uint32_t topLeft = stack * ringStride + slice;
63             const std::uint32_t bottomLeft = topLeft + ringStride;
64             for (const std::uint32_t index : std::array{topLeft, bottomLeft, topLeft + 1
U, topLeft + 1U, bottomLeft, bottomLeft + 1U}) {
65                 indices.push_back(index);
66             }
67         }
68     }
69 }
70
```

59 | ringStride

Each ring contains slices+1 positions. Adding this stride moves from one ring to the same slice on the next ring.

60-64 | Two triangles per cell

topLeft and bottomLeft identify a cell. The temporary std::array lists six indices: three for each of its two triangles. The index order determines which vertices form each face.

65 | Append indices

push_back stores each integer in the output vector. The triangles reuse already generated positions instead of duplicating every vertex per triangle.

69 | Function completion

No value is returned because both vectors were filled through references. With 24 x 32 cells, the array contains 4,608 indices, describing 1,536 triangles; pole cells include degenerate triangles.

loadScene: create the repeated ground plane

game.cpp | source lines 71-84 | explanations refer to these numbers

```
71 /// @brief Builds the grassy ground, the sun, and the sun's directional light plus ambie
    nt.
72 /// @return The sun's render-object handle so the frame loop can keep it anchored to the
    camera.
73 [[nodiscard]] std::expected<vve::RenderObjectHandle, vve::Error> loadScene(vve::RenderSy
    stem render,
74                                     const std::filesystem::path &root) {
75     render.clearScene();
76
77     // One quad covers the field; UVs span 0..10 and the material sampler repeats the gr
    ass.
78     const auto grassTexture = root / grassTextureRelativePath;
79     constexpr float textureRepeats = (groundHalfExtent * 2.0F) / groundTileSize;
80     if (auto result = render.addTexturedPlane(vve::Vec2{groundHalfExtent, groundHalfExte
    nt},
81         grassTexture, vve::Vec2{textureRepeats, textureRepeats}); !result) {
82         return std::unexpected(result.error());
83     }
84 }
```

73-74 | Parameters and result

This application helper receives a `RenderSystem` wrapper by value and a filesystem path by const reference. Copying the wrapper still reaches the same engine-owned renderer. `expected<RenderObjectHandle, Error>` eventually returns the sun handle or an error.

75-79 | Clear and prepare

`clearScene` removes existing render content. `filesystem::path` operator `/` joins the asset root and grass filename. `textureRepeats = 40 / 4 = 10`; it is calculated at compile time.

80-81 | addTexturedPlane

The two `Vec2` values describe half extent (20,20) and UV scale (10,10). This creates one XZ quad with one repeated texture. The renderer uses repeat addressing outside UV range 0..1; it does not build ground from tiled boxes.

82 | Propagate failure

`expected` is checked before use. `unexpected(result.error())` forwards the error to main. The successful plane handle is not retained because this game never moves or removes the ground individually.

Two different loadScene names

This helper creates procedural objects. `AssetSystem::loadScene(path)`, used by other examples, imports a model file. That asset-import call does not occur in this game.

loadScene: create and configure the sun mesh

game.cpp | source lines 85-100 | explanations refer to these numbers

```
85     // The sun is a bright sphere drawn in a constant color; the frame loop pins it far
    away relative to the camera.
86     vve::Vector<vve::Vec3> sunPositions{};
87     vve::Vector<std::uint32_t> sunIndices{};
88     makeSphere(sunRadius, 24U, 32U, sunPositions, sunIndices);
89     const auto sun = render.addTriangleMesh(std::move(sunPositions), std::move(sunIndices),
90     vve::LinearColor{.value = vve::Vec3{1.0F, 0.92F, 0.35F}});
91     if (!sun) {
92         return std::unexpected(sun.error());
93     }
94     // The sun is the light source itself: flat emissive shading and no shadow thrown on
    to the field.
95     if (auto result = render.setObjectUnlit(*sun, true); !result) {
96         return std::unexpected(result.error());
97     }
98     if (auto result = render.setObjectCastsShadow(*sun, false); !result) {
99         return std::unexpected(result.error());
100 }
```

86-88 | Temporary CPU geometry

sunPositions and sunIndices are empty vectors. makeSphere fills them using radius 1.6, 24 stacks and 32 slices. They are geometry data, not render objects yet.

89-90 | addTriangleMesh

std::move transfers the vectors into the by-value call. LinearColor wraps the yellow RGB value. The result identifies one render object; GPU resources are prepared by the engine as rendering requires them.

91-96 | Result and unlit appearance

A failed mesh creation returns its Error. After success, *sun extracts the handle. setObjectUnlit(handle,true) makes the visible sphere ignore scene lighting so it stays bright. It does not itself create illumination or global light emission.

98-100 | Shadow setting

setObjectCastsShadow(handle,false) excludes this sphere from shadow casting. Both configuration calls are checked and propagate errors. The separate directional-light call on the next page supplies illumination.

loadScene: illuminate the scene and return

game.cpp | source lines 101-113 | explanations refer to these numbers

```
101
102     // The sun casts a warm directional light straight down onto the field, with a touch
    of sky ambient.
103     render.setDirectionalLight(vve::Direction{.value = sunLightDirection},
104                               vve::LinearColor{.value = vve::Vec3{1.0F, 0.96F, 0.85F}}, vve::LightInte
nsity{.value = 1.35F},
105                               vve::LinearColor{.value = vve::Vec3{0.32F, 0.34F, 0.40F}});
106     return *sun;
107 }
108
109 } // namespace
110
111 /**
112  * @brief Runs the crate-collector game using only the public VVE facade.
113  */
```

103-105 | setDirectionalLight

Direction wraps the light direction; LinearColor wraps warm light RGB and the ambient RGB request; LightIntensity supplies 1.35. A directional light has a direction rather than a local position. The bright sphere is its visual marker, not the light resource.

106 | Return the sun ID

*sun extracts the valid RenderObjectHandle and constructs a successful expected result. main keeps this handle so each frame can reposition the sphere relative to the camera.

107-109 | End the helper scope

The temporary vectors and wrapper copy leave scope. RenderSystem owns the scene resources, so the created plane and sun remain. The anonymous namespace ends; these helpers remain private to the file.

111-113 | Next entry point

The next function is main. Reading the file top to bottom defines helpers first; execution begins in main and reaches loadScene only when main calls it.

main: diagnostics and command-line options

game.cpp | source lines 114-133 | explanations refer to these numbers

```

114 int main(int argc, char **argv) {
115     std::cout << std::unitbuf;
116     std::cerr << std::unitbuf;
117     std::cout << "[game] engine=" << vve::engineImplementationNamespaceName << '\n';
118     int selectedTrack{};
119     bool requireAudio{};
120     for (int argument = 1; argument < argc; ++argument) {
121         const std::string_view option{argv[argument]};
122         if (option == "--require-audio") { requireAudio = true; }
123         if (option == "--music-track") {
124             if (++argument >= argc) { std::cerr << "--music-track needs 0, 1 or 2.\n"; r
return 4; }
125             const std::string_view value{argv[argument]};
126             const auto parsed = std::from_chars(value.data(), value.data() + value.size(
), selectedTrack);
127             if (parsed.ec != std::errc{} || parsed.ptr != value.data() + value.size() ||
128                 selectedTrack < 0 || selectedTrack >= static_cast<int>(musicFiles.size())
)) {
129                 std::cerr << "--music-track needs 0, 1 or 2.\n"; return 4;
130             }
131         }
132     }
133 }

```

114-119 | Entry and defaults

argc/argv carry command-line arguments. std::unitbuf flushes cout and cerr after writes, making logs visible promptly. The implementation name is printed. selectedTrack defaults to 0 (Dance); requireAudio defaults to false.

120-126 | Read option text

string_view observes each argument without owning a copy. --require-audio makes audio required. --music-track consumes the next argument. data and size provide the character range for from_chars, which parses an integer without throwing.

127-130 | Validate the whole value

The code rejects parsing errors, leftover characters and values outside 0..2. Incrementing argument skips the already consumed value. Invalid selection returns exit code 4.

Other options

This loop ignores unrelated options. --frames is handled later by ExampleSupport::frameLimit. The music test flags exercise startup without adding a second game implementation.

main: build the engine and initialize windows

game.cpp | source lines 134-149 | explanations refer to these numbers

```
134     const auto activeRenderer = vve::RendererId{.value = "forward"};
135     auto engine = vve::EngineBuilder<>{}
136         .applicationName("game")
137         .addWindow(vve::WindowSetup{}
138             .id("main")
139             .title("VVE Crate Collector")
140             .extent(vve::PixelExtent{.width = windowWidth, .height = windowHeight})
141             .renderer(activeRenderer)
142             .resizable(true))
143         .build();
144
145     if (const auto result = engine.init(); !result) {
146         std::cerr << "[game] engine init failed: error=" << vve::errorName(result.error(
147     )) << '\n';
148         return 1;
149     }
```

134-135 | `RendererId` and `EngineBuilder`

`RendererId` names the forward renderer. `EngineBuilder<>` gathers startup settings; empty `<>` means no registered user systems. The game implements its ongoing rules directly in `main`.

136-142 | `WindowSetup` and fluent calls

`applicationName` names the application. `WindowSetup` describes the main window: lookup ID, visible title, `PixelExtent` size, renderer and resize permission. Each method changes its helper and returns it for the next call; it does not open a window immediately.

143 | `build()`

`build` constructs `vve::Engine<>`. It owns the selected implementation and connects facade wrappers to its subsystems. Temporary builder/window descriptors end after construction; the Engine keeps the startup values.

145-148 | `init()`

`init` starts the simple engine platform/windows and any user initialization hooks. There are none here. The renderer initializes GPU resources lazily on the first drawable step. A failed `init` is printed with `errorName` and returns exit code 1.

main: get services and load the soundtrack

game.cpp | source lines 150-170 | explanations refer to these numbers

```

150     auto render = engine.world().get<vve::RenderSystem>();
151     const auto root = vve::example::assetRoot(argc > 0 ? argv[0] : nullptr);
152     // The public facade owns sounds; one looped voice plays the selected V2 track.
153     auto audio = engine.world().get<vve::AudioSystem>();
154     bool musicReady = audio.init().has_value();
155     float musicVolume{0.35F};
156     bool musicPaused{};
157     std::array<vve::SoundHandle, musicFiles.size()> songs{};
158     vve::AudioPlaybackHandle musicVoice{};
159     const auto musicDirectory = std::filesystem::absolute(argv[0]).parent_path() / "audi
o/v2";
160     if (musicReady) {
161         for (std::size_t index = 0; index < musicFiles.size(); ++index) {
162             const auto sound = audio.loadSound(musicDirectory / musicFiles[index], vve::
AudioLoadMode::on_demand);
163             if (!sound) { std::cerr << "[game] music load failed: " << audio.lastError()
<< '\n'; return 4; }
164             songs[index] = *sound;
165         }
166         if (!audio.setMasterVolume(vve::AudioVolume{.value = musicVolume})) { return 4;
}
167     } else {
168         std::cerr << "[game] audio unavailable: " << audio.lastError() << '\n';
169         if (requireAudio) { return 4; }
170     }

```

150-153 | World and facade copies

engine.world() creates a temporary reference view. get<T>() returns a service wrapper stored by the engine. auto makes local RenderSystem and AudioSystem wrapper copies; they still reach the same implementation objects and must not outlive the Engine.

151,159 | filesystem paths

assetRoot searches the working directory, executable ancestors and configured source root for assets. Audio uses absolute(argv[0]).parent_path()/audio/v2, the music directory beside the supplied executable path. path objects describe locations; they do not load files.

154-158 | Audio state and handles

audio.init opens the output device separately from engine.init. musicReady records success. The sound-handle std::array keeps reusable tracks; musicVoice identifies the current playback occurrence. Volume is 0.35, and music starts unpaused.

160-166 | Load and set gain

loadSound returns expected<SoundHandle,Error>. on_demand retains encoded data and decodes during playback; it is not disk streaming. *sound is saved after checking. AudioVolume names the master gain, which multiplies all voice gains.

167-170 | Optional output device

An unavailable device is logged; the game continues unless requireAudio is true. Loading or initial playback failures after device initialization instead return code 4. lastError supplies the mixer diagnostic.

main: define music switching and build the scene

game.cpp | source lines 171-190 | explanations refer to these numbers

```

171     auto startMusic = [&]() -> std::expected<void, vve::Error> {
172         if (musicVoice.valid()) {
173             if (const auto result = audio.stop(musicVoice); !result) { return result; }
174             musicVoice = {};
175         }
176         const auto voice = audio.play(songs[selectedTrack], vve::AudioPlaybackOptions{.1
oop = true});
177         if (!voice) { return std::unexpected(voice.error()); }
178         musicVoice = *voice;
179         std::cout << "[game] music=" << musicFiles[selectedTrack] << " loop=1\n";
180         return musicPaused ? audio.pause(musicVoice) : std::expected<void, vve::Error>{}
;
181     };
182     if (musicReady && !startMusic()) { std::cerr << "[game] music failed: " << audio.las
tError() << '\n'; return 4; }
183     const auto sunHandle = loadScene(render, root);
184     if (!sunHandle) {
185         std::cerr << "[game] scene load failed: error=" << vve::errorName(sunHandle.erro
r()) << '\n';
186         return 2;
187     }
188     // Direction from any viewpoint toward the sun: opposite to the light it sends down.
189     const auto toSun = vve::math::normalize(vve::math::scale(sunLightDirection, -1.0F));
190

```

171 | The startMusic lambda

[&] captures surrounding state by reference. The closure returns `expected<void, Error>` and is called now and later by the GUI. It controls audio, not a new engine. All referenced state stays alive throughout the loop.

172-178 | Replace one playback voice

valid checks for a nonzero handle. stop releases the previous voice; {} resets the local ID. play uses a retained `SoundHandle` and `AudioPlaybackOptions{loop=true}`, creating a fresh independently controlled voice. A failed call propagates its `Error`.

179-182 | Preserve pause and start

The log names the song. If `musicPaused` is true, the new voice is paused immediately; otherwise an empty `expected<void, Error>` means success. The startup call runs only when the device is ready.

183-187 | Call the scene helper

`loadScene` creates the ground and sun already explained. `sunHandle` is an `expected` containing the ID used later. Failure returns code 2; resources already owned by the Engine are cleaned up on exit.

189 | Direction toward the sun

scale multiplies the light-travel vector by -1; `normalize` gives a unit vector pointing from the camera toward the visible sun.

main: initialize the camera and game state

game.cpp | source lines 191-205 | explanations refer to these numbers

```

191     // Start the camera on the plane, at eye height, looking toward the centre of the fi
    eld.
192     vve::DefaultCameraController cameraController{};
193     cameraController.eye = vve::Position{.value = vve::Vec3{0.0F, eyeHeight, 14.0F}};
194     const auto startupForward =
195         vve::math::normalize(vve::math::subtract(vve::Vec3{0.0F, eyeHeight, 0.0F}, camer
    aController.eye.value));
196     cameraController.yaw = std::atan2(startupForward.x, -startupForward.z);
197     cameraController.pitch = std::asin(startupForward.y);
198
199     std::vector<Crate> crates{};                ///< Active crates currently in t
    he world.
200     int score{};                                ///< Crates collected so far.
201     std::size_t spawnedCrates{};                ///< Crates spawned so far; selec
    ts the crate texture.
202     float spawnTimer{spawnInterval};            ///< Time accumulator that spawns
    the first crate immediately.
203     std::mt19937 rng{std::random_device{}()};    ///< Random source for crate spaw
    n positions.
204     std::uniform_real_distribution<float> place{-groundHalfExtent + 2.0F, groundHalfExte
    nt - 2.0F};
205

```

192-197 | DefaultCameraController

This persistent controller stores eye position, yaw, pitch and movement settings. Position wraps the starting eye (0,1,14). subtract makes a direction toward (0,1,0); normalize gives unit length. atan2 and asin convert that direction into controller angles.

199-202 | State kept between frames

std::vector<Crate> owns the active records. score counts collections; spawnedCrates chooses alternating textures. spawnTimer begins at spawnInterval, so the first loop can spawn immediately. These local values are also read by the GUI callback.

203 | Random generator

std::random_device supplies a seed to std::mt19937. The Mersenne Twister then keeps its random state. Spawn positions vary between runs; this example is not seeded deterministically.

204 | Random distribution

uniform_real_distribution<float> maps generator values into approximately -18..18. Calling place(rng) later picks X and Z positions, keeping crate centers inside the ground boundary.

No camera resource allocation here

The controller and vector/angle values describe a view. setCamera in the frame loop publishes a Camera value to RenderSystem.

main: register the score and music GUI

game.cpp | source lines 206-230 | explanations refer to these numbers

```

206     // The score overlay is drawn every frame in the top-left corner.
207     engine.world().get<vve::GuiSystem>().draw([&] {
208         ImGui::SetNextWindowPos(ImVec2(12.0F, 12.0F), ImGuiCond_Always);
209         ImGui::Begin("Score", nullptr,
210             ImGuiWindowFlags_NoMove | ImGuiWindowFlags_NoResize | ImGuiWindowFlags_A
lwaysAutoResize);
211         ImGui::Text("Score: %d", score);
212         ImGui::Text("Crates on field: %zu", crates.size());
213         ImGui::Separator();
214         ImGui::TextUnformatted("WASD move - Arrows look - Esc quit");
215         ImGui::TextUnformatted("Drive into a crate to collect it!");
216         ImGui::Separator();
217         ImGui::BeginDisabled(!musicReady);
218         if (ImGui::Combo("Music", &selectedTrack, "Dance\0Ophelia\0Never get out\0")) {
219             musicReady = startMusic().has_value();
220         }
221         if (ImGui::SliderFloat("Volume", &musicVolume, 0.0F, 1.0F, "%.2f")) {
222             musicReady = audio.setMasterVolume(vve::AudioVolume{.value = musicVolume}).h
as_value();
223         }
224         if (ImGui::Checkbox("Pause music", &musicPaused)) {
225             musicReady = (musicPaused ? audio.pause(musicVoice) : audio.resume(musicVoic
e)).has_value();
226         }
227         ImGui::EndDisabled();
228         if (!musicReady) { ImGui::TextUnformatted("Audio unavailable"); }
229         ImGui::End();
230     });

```

207-210 | draw registers a callback

GuiSystem::draw stores this lambda; it runs during prepared GUI frames inside engine.step. [&] refers to persistent locals. SetNextWindowPos constructs an ImVec2 location. Begin opens the Score panel; flags fix its position/size while auto-sizing its contents.

211-217 | Text and enabled state

Text formats score and crates.size(); TextUnformatted draws fixed instructions. Separator divides controls. BeginDisabled prevents music interaction when initialization or a control operation has failed.

218-223 | Choose a song or gain

Combo edits selectedTrack and returns true on a change; the nul-separated string provides three labels. Then startMusic replaces playback. SliderFloat edits musicVolume; a change calls setMasterVolume with AudioVolume and records success through has_value.

224-230 | Pause and close scopes

Checkbox edits musicPaused; its change calls pause or resume on musicVoice. EndDisabled closes the disabled scope, unavailable text reports a failure state, and End closes the window. The lambda ends here but remains registered for subsequent frames.

Frame loop: time, input, camera and sun

game.cpp | source lines 231-254 | explanations refer to these numbers

```

231
232     const int maxFrames = vve::example::frameLimit(argc, argv);
233     int frame{};
234     bool running = true;
235     auto lastTime = std::chrono::steady_clock::now();
236
237     while (running && (maxFrames == 0 || frame < maxFrames)) {
238         const auto now = std::chrono::steady_clock::now();
239         const float dt = std::clamp(std::chrono::duration<float>(now - lastTime).count()
, 0.0F, 0.1F);
240         lastTime = now;
241
242         // Steer with the standard controller but pin the eye to a fixed height so it stays on the plane.
243         const auto input = engine.world().get<vve::WindowSystem>().input();
244         cameraController.eye.value.y = eyeHeight;
245         const auto steered = cameraController.update(input, vve::DeltaTime{dt});
246         cameraController.eye.value.y = eyeHeight;
247         const auto eyePosition = cameraController.eye;
248         const auto camera = vve::Camera::lookAt(
249             eyePosition, vve::Position{.value = vve::math::add(eyePosition.value, steered.forward.value)});
250         render.setCamera(camera);
251         // Keep the sun at a fixed far offset from the eye so it shows no parallax as the camera moves.
252         (void)render.setObjectTransform(*sunHandle, vve::Transform{.translation = vve::Position{
253             .value = vve::math::add(eyePosition.value, vve::math::scale(toSun, sunDistance))}});
254

```

232-240 | Frame limit and timing

frameLimit reads --frames; zero means no cap. steady_clock is monotonic. duration<float> converts elapsed time to seconds; clamp limits dt to 0..0.1, reducing large jumps after a stall. This dt belongs to the game, not engine.frameContext.

243-246 | InputState and steering

input is a live facade view. Before step, its key queries see the previous event poll. DefaultCameraController::update applies movement/rotation using DeltaTime. Eye Y is reset before and after to keep the player at height 1.

247-250 | Camera::lookAt and setCamera

eyePosition copies the current Position. add combines that eye with the steered forward vector to create a target point. lookAt makes a fresh Camera; setCamera copies it into the renderer. The view keeps default clipping and field of view.

251-253 | Transform the sun

scale gives the 80-unit offset; add anchors it to the eye. Position and Transform describe that placement. setObjectTransform updates the render object. The explicit (void) discards this expected result. Moving the sun with the eye removes camera-motion parallax.

Frame loop: spawn a visible crate

game.cpp | source lines 255-272 | explanations refer to these numbers

```

255         // Pause the timer at capacity so a collected crate waits for the normal respawn
interval.
256         if (crates.size() < maxActiveCrates) {
257             spawnTimer += dt;
258             if (spawnTimer >= spawnInterval) {
259                 spawnTimer = 0.0F;
260                 const vve::Vec3 spawnPosition{place(rng), crateSpawnY, place(rng)};
261                 const vve::Vec3 minimum{-crateHalfSize, -crateHalfSize, -crateHalfSize};
262                 const vve::Vec3 maximum{crateHalfSize, crateHalfSize, crateHalfSize};
263                 const auto crateTexture = root / crateTextureRelativePaths[spawnedCrates
++ % crateTextureRelativePaths.size()];
264                 if (auto added = render.addTexturedCuboid(minimum, maximum, crateTexture
,
265                     vve::Transform{.translation = vve::Position{.value = spawnPosition}});
266                     added) {
267                     crates.push_back(
268                         Crate{.handle = *added, .position = spawnPosition, .velocityY = 0.0F, .l
anded = false});
269                 }
270             }
271         }
272

```

256-259 | Capacity and interval

The timer advances only below the cap. At the interval it resets to zero; this spawns at most one crate per iteration. Excess elapsed time is discarded rather than producing catch-up spawns.

260-263 | Position, local bounds and texture

Two place(rng) calls choose X/Z; Y is 24. Vec3 minimum/maximum describe a unit cube centered at its own origin. spawnedCrates++ modulo array size alternates wood textures; path / resolves the selected image.

264-266 | addTexturedCuboid

Transform{translation=Position{spawnPosition}} places the local cube in world space. The engine creates a visible textured object and returns expected<RenderObjectHandle,Error>. The if condition proceeds only on success.

267-269 | Instantiate Crate and store it

Crate{...} ties that render handle to the spawn position, zero velocity and airborne flag. push_back retains the record for gravity and collection. It may grow the vector; no persistent references to its elements are kept here.

Failure behavior

Failed cuboid creation adds no Crate and is not logged here. The timer and texture counter have already advanced; the next ordinary interval can try again.

Frame loop: gravity and landing

game.cpp | source lines 273-288 | explanations refer to these numbers

```
273      // Integrate gravity for airborne crates and rest them on the ground when they l
and.
274      for (auto &crate : crates) {
275          if (crate.landed) {
276              continue;
277          }
278          crate.velocityY -= gravity * dt;
279          crate.position.y += crate.velocityY * dt;
280          if (crate.position.y <= crateRestY) {
281              crate.position.y = crateRestY;
282              crate.velocityY = 0.0F;
283              crate.landed = true;
284          }
285          (void)render.setObjectTransform(crate.handle,
286              vve::Transform{.translation = vve::Position{.value = crate.position}});
287      }
288
```

274-277 | Iterate live state by reference

auto& crate reaches each stored record without copying it. continue skips landed crates so they stay at rest.

278-279 | Advance velocity, then position

Subtracting gravity*dt accelerates downward. Adding the updated velocity*dt changes height. This is semi-implicit Euler integration performed by the game. It is not a call to a rigid-body physics solver.

280-284 | Ground contact rule

A center at or below y=0.5 is clamped to 0.5, speed is zeroed and landed becomes true. The plane stays at y=0, so this unit cube rests on it.

285-286 | Publish the result

Construct a Transform containing Position{crate.position}, then setObjectTransform on the retained render handle. Rendering now has the updated placement for the upcoming step. The code discards the expected result with (void); robust game code can inspect it.

What remains unchanged?

X and Z do not move during falling. There is no crate-crate collision, bouncing or horizontal simulation in this example.

Frame loop: engine.step and stopping

game.cpp | source lines 289-302 | explanations refer to these numbers

```
289         const auto status = engine.step();
290         if (!status) {
291             std::cerr << "[game] frame failed: error=" << vve::errorName(status.error())
<< '\n';
292             return 3;
293         }
294         ++frame;
295         if (requireAudio && !musicReady) { std::cerr << audio.lastError() << '\n'; return
n 4; }
296         if (*status == vve::FrameStatus::stopped) {
297             break;
298         }
299         if (input.wasKeyPressed(vve::Key::escape)) {
300             running = false;
301         }
302
```

289 | step enters the engine

The public facade polls the implementation, obtains frame context, calls registered user systems, then renders. EngineBuilder<> registered none, so the game rules have already run in main. First drawing initializes Vulkan resources and GUI as required.

What runs behind this call?

The simple engine collects finished audio voices and polls SDL events. The renderer uploads/synchronizes scene changes, records GPU drawing, prepares the GUI callback and presents drawable targets. The mixer device produces audio independently.

290-298 | Expected versus FrameStatus

Failure prints errorName and returns code 3. A successful call increments the local frame count. Required-audio mode turns a GUI music failure into code 4. A stopped status is a normal result; break exits without running collection for that iteration.

299-301 | Escape uses fresh input

InputState still references the same implementation; after step its queries see the newly polled events. wasKeyPressed tests the press edge. Setting running=false stops the next iteration, but this iteration still reaches collection.

Frame loop: collect crates and exit

game.cpp | source lines 303-324 | explanations refer to these numbers

```

303      // Collect any landed crate the camera has reached, removing it and scoring a po
      int.
304      const auto eye = cameraController.eye.value;
305      std::erase_if(crates, [&](const Crate &crate) {
306          if (!crate.landed) {
307              return false;
308          }
309          const float dx = eye.x - crate.position.x;
310          const float dz = eye.z - crate.position.z;
311          if ((dx * dx + dz * dz) > (collectRadius * collectRadius)) {
312              return false;
313          }
314          if (render.removeObject(crate.handle)) {
315              ++score;
316              return true;
317          }
318          return false;
319      });
320  }
321
322  std::cout << "[game] frames=" << frame << " score=" << score << '\n';
323  return 0;
324  }

```

304-308 | erase_if and its predicate

eye copies the controller position. `std::erase_if` examines each `Crate` with a lambda; true means erase that game record. Unlanded crates return false. [&] lets the predicate reach render, score and eye.

309-313 | Horizontal distance test

dx and dz compare the player and crate centers in the XZ plane. Squared distance is compared with squared radius, avoiding sqrt. A crate outside radius 1.2 is retained.

314-319 | Remove the picture and the record

`removeObject` is checked first. Only successful removal increments score and returns true, causing `erase_if` to erase the record. Failure returns false so its record remains available.

Collection follows drawing

The preceding step already drew the crate and old score. Removal and score changes normally appear on the next rendered frame. The engine owns render-resource lifetime and safely releases eligible unused resources.

322-324 | Exit and automatic cleanup

`cout` prints frames and score; return 0 ends normal execution. Local game state is destroyed and the Engine releases its audio, GUI, rendering and window resources. The callback is never invoked again during this normal teardown.

CMake: compile and link the executable

CMakeLists.txt | source lines 1-22 | explanations refer to these numbers

```
1  add_executable(game
2      game.cpp
3  )
4
5  target_link_libraries(game PRIVATE
6      ViennaVulkanEngine::ViennaVulkanEngine
7      vve_example_support
8  )
9
10 add_dependencies(game
11     ViennaVulkanEngine
12     vve_simple_shaders
13 )
14
15 vve_set_output_dirs(game)
16
17 # Keep V2's soundtrack and attribution beside the executable. These are build
18 # operations; the game opens the files itself without a launch script.
19 set(game_music_files)
20 foreach(name dance.mp3 ophelia.mp3 getout.ogg license.txt V2_MUSIC.md)
21     list(APPEND game_music_files "${PROJECT_SOURCE_DIR}/assets/sounds/${name}")
22 endforeach()
```

1-3 | add_executable

Creates the game target from game.cpp. The source imports modules; CMake supplies the compiler commands and compiled-module dependencies.

5-8 | target_link_libraries

ViennaVulkanEngine::ViennaVulkanEngine supplies the engine implementation, public module settings and linked dependencies. vve_example_support supplies the asset/frame helpers. PRIVATE means these dependencies belong to this executable target.

10-15 | Prerequisites and output

add_dependencies makes the engine and generated simple shaders ready before this target. vve_set_output_dirs puts the executable in bin/debug/exe or bin/release/exe and arranges engine runtime support.

17-22 | A list of runtime files

set creates an empty CMake list. foreach visits the three music files and two credit documents; list(APPEND) makes absolute source paths under assets/sounds. This is build-time file preparation, not C++ playback.

Build-file context

This file is included by the engine example build. It relies on targets and functions already defined by the parent; it is not a standalone top-level CMake project.

CMake: copy music and register checks

CMakeLists.txt | source lines 23-37 | explanations refer to these numbers

```

23 add_custom_target(game_audio_assets ALL
24     COMMAND "${CMAKE_COMMAND}" -E make_directory "${TARGET_FILE_DIR:game}/audio/v2"
25     COMMAND "${CMAKE_COMMAND}" -E copy_if_different
26         ${game_music_files} "${TARGET_FILE_DIR:game}/audio/v2"
27     VERBATIM)
28 add_dependencies(game game_audio_assets)
29
30 if(BUILD_TESTING)
31     foreach(track RANGE 0 2)
32         add_test(NAME GameMusic${track} COMMAND game --frames 3 --music-track ${track} -
33             -require-audio)
34         set_tests_properties(GameMusic${track} PROPERTIES
35             ENVIRONMENT "SDL_AUDIO_DRIVER=dummy" LABELS "gpu;audio;example" TIMEOUT 60
36             WORKING_DIRECTORY "${TARGET_FILE_DIR:game}")
37     endforeach()
38 endif()

```

23-27 | game_audio_assets

add_custom_target(... ALL) runs music preparation on ordinary builds. cmake -E make_directory creates audio/v2 beside the executable. copy_if_different copies the listed music/credits only when needed. TARGET_FILE_DIR:game resolves the actual output directory; VERBATIM preserves arguments.

28 | Order asset preparation

add_dependencies(game game_audio_assets) ensures this preparation is part of building game. No launch script is needed to copy music or start playback; the executable loads the files itself.

30-37 | CTest startup checks

When BUILD_TESTING is enabled, foreach creates GameMusic0, GameMusic1 and GameMusic2. Each launches this same executable for three frames with the selected track and required audio.

33-35 | Test properties

SDL_AUDIO_DRIVER=dummy supplies a device without physical speakers. Labels group GPU/audio/example tests. TIMEOUT prevents an indefinitely running test. WORKING_DIRECTORY points to the executable folder. These settings are for tests, not the normal player launch.

Expected runtime package

Keep game.exe, its configuration-matched DLLs, and audio/v2 music/credits together. The ground/crate image search still uses assetRoot; this target copies the soundtrack, not every graphic asset.